4. Understand the Dataset

The file **data/auth_logs.csv** contains fictional authentication events. Each row represents one login event.

Field	Meaning	Example
timestamp	Date and time of the event.	2026-10-27T09:02:11
username	Account associated with the event.	maya
source_ip	Source IP recorded for the event.	192.0.2.20
event	Type of event.	login
result	Authentication outcome.	failure or success

Example row

```
2026-10-27T09:02:11,maya,192.0.2.20,login,failure
```

This row establishes that a login failure was recorded for maya from 192.0.2.20 at that time. It does not establish why it failed or who controlled the source.

5. Task 6 — Manual Log Investigation

Goal: identify patterns and explain what should be checked next. Avoid declaring an attack without corroborating evidence.

Investigation questions

- Are there repeated failed logins from one IP?
- Does a successful login follow repeated failures for the same account and IP?
- Does one IP attempt logins for several usernames?
- Are successful logins outside the assumed normal-hours window?

Evidence	Possible interpretation	Follow-up
Repeated failures from one IP.	Password mistakes or guessing activity.	Check account, MFA, source ownership, and rate limiting.
Success after repeated failures.	Could be legitimate recovery or successful guessing.	Verify with user; review session/device records.
One IP touches several usernames.	Could be enumeration or a shared VPN/NAT source.	Confirm network ownership and expected usage.
Successful login at an unusual hour.	May be unusual for that account.	Compare schedule and approved remote access.

6. Interpret Findings Correctly

- **Fact:** directly visible in logs, such as five failure events from one IP.
- **Hypothesis:** one possible explanation, such as password guessing.
- **Conclusion:** a judgment supported by enough corroborating evidence. A small authentication log may not justify a definitive conclusion.

Recommended investigation actions

- Check whether IPs belong to an approved VPN, NAT, campus, or corporate egress address.
- Review MFA results, session creation, device identifiers, and endpoint alerts if available.
- Ask the account owner to verify unexpected successful logins.
- Preserve relevant logs and record the reason for follow-up.
- Recommend rate limiting and suitable authentication protections where justified.

7. Task 13 — How the Python Tool Works

The file **src/log_analyzer.py** reads the CSV, validates required columns and values, sorts events by timestamp, evaluates detection rules, and writes a JSON report. It uses Python's standard library.

Rule	Trigger condition in this prototype	Meaning
Repeated failures	At least 5 failed login events from one IP.	Flag for investigation.
Success after failures	At least 3 earlier failures for the same username/IP before a success.	Verify account and session.
Multiple accounts	One IP has activity for at least 3 distinct usernames.	Check shared-source behavior or enumeration.
After-hours success	Successful login outside 06:00–22:00.	Compare with expected schedule.

These are demonstration rules. They do not calculate a time-windowed rate or use organization-specific baselines. Thresholds can create false positives and must be tuned before real deployment.

8. Run the Project in VS Code on Windows

Step 1 — Extract the project ZIP and open the extracted folder in VS Code. Open Terminal → New Terminal. The terminal should be at the project root (the folder containing README.md).

Step 2 — Check Python

```
python --version
```

Step 3 — Run the analyzer

```
python src/log_analyzer.py --input data/auth_logs.csv --output reports/analysis_results.json
```

The console prints summary counts and the path to the JSON report. Open **reports/analysis_results.json** to inspect the detailed alerts.

Step 4 — Run the tests

```
python -m unittest discover -s tests -v
```

The included tests cover the repeated-failure threshold, success after three failures, and a below-threshold case. Run them yourself and retain the actual output; do not claim tests passed unless you have executed them.

Step 5 — Capture your own screenshots

- Terminal showing the analyzer command and summary.
- Generated JSON report open in VS Code.
- Terminal showing the test command and actual results.
- Optional screenshot of the source code's detection logic.

9. Test Plan

ID	Scenario	Expected result
T1	Five failed events from one IP.	Repeated-failed-logins alert.
T2	Three failures then success for same user/IP.	Success-after-failures alert.
T3	Only two failures from one IP.	No repeated-failure threshold alert.
T4	Successful login at 23:48 in sample data.	After-hours triage alert.
T5	Missing required CSV column or invalid timestamp.	Input validation error.

10. Expected Project Files

File	Purpose
README.md	Project summary, setup, commands.
data/auth_logs.csv	Synthetic source dataset.
src/log_analyzer.py	Python implementation.
tests/test_log_analyzer.py	Automated unit tests.
reports/analysis_results.json	Generated alert output.
reports/sample_run_output.txt	Saved sample console output.
reports/test_results.txt	Saved test output; verify locally.
reports/task6_findings.md	Task 6 evidence and investigation report.
reports/task13_tool_report.md	Task 13 implementation and test report.

11. Limitations and Responsible Use

- The data is synthetic and small; it may not represent real organizational behavior.
- Example IPs are not evidence of real malicious infrastructure.
- The prototype does not prove compromise, attribute an actor, or automatically respond.
- Business hours and thresholds are illustrative.
- Real deployment needs access controls, data protection, configurable rules, and organization-specific validation.

12. Final Submission Checklist

- Run the analyzer and tests on your own computer.
- Review the JSON and verify alerts against the CSV rows.
- Personalize reports with your name and required academic details.
- Add your own screenshots and accurately record test results.
- Keep Task 6 findings and Task 13 implementation as separate deliverables.
- Upload to GitHub or Google Drive and test the link in a private/incognito browser window.
- Submit both tasks through the designated Google Form before the deadline.

13. Short Explanation for the Evaluator

“For Task 6, I analyzed synthetic authentication logs and documented suspicious patterns while separating evidence from assumptions. For Task 13, I implemented a Python command-line analyzer that checks the same data against four defensive rules, produces JSON alerts, and includes unit tests. The alerts are triage indicators, so I recommend checking MFA, session, device, and network context before deciding whether an incident occurred.”

14. Alignment with the Assignment

The assessment catalogue defines Task 6 as security-log analysis with annotated findings and justified observations, and Task 13 as a defensive prototype with sample output and test cases. This guide organizes the workflow around those deliverables. Follow any additional instructions from your instructor or assessment portal.

End of workflow guide